MODULE 8 · PACKAGING & EXTENSIBILITY

Helm

Templated packages, releases, and safe rollbacks

WHY IT MATTERS

An app is more than one manifest

Kustomize (Lab 8.1) patches YAML you already own. Helm goes further: it packages a whole application — Deployment, Service, config, and more — into one versioned, parameterized bundle called a **chart**.

Instead of hand-editing manifests per environment, you set **values** and Helm renders the templates for you — then tracks the result as a **release** you can upgrade or roll back.

BY THE END YOU CAN

- Explain chart / release / revision
- Install, upgrade, and rollback a release
- Override values with `--set` and a values file
- Say when Helm beats Kustomize (and vice versa)

CONCEPT

Chart + values render into a release

A **chart** is a directory of Go-templated manifests plus a `values.yaml` of defaults. This lab uses a **local chart** shipped with it (`webapp-chart/`) — no external chart repo to reach.

Helm merges your overrides into those defaults, renders every template into plain Kubernetes YAML, and applies it as a tracked **release**.

Nothing here needs a registry or the internet — the chart is just a folder on disk.

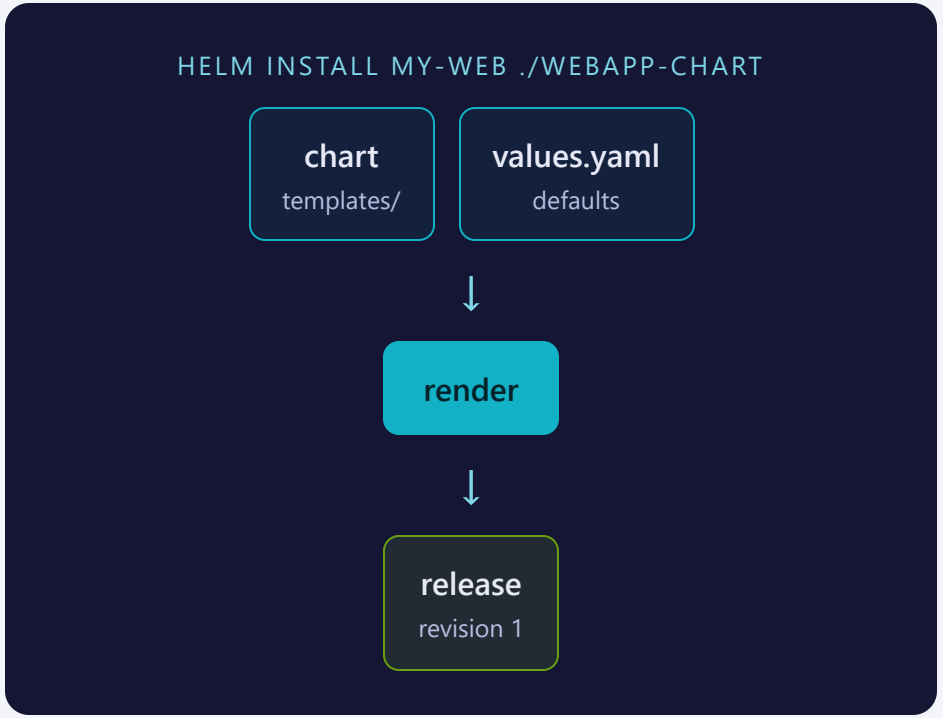


CHART ANATOMY

Four files, one convention

FILE	PURPOSE
`Chart.yaml`	Metadata — name, chart version, app version
`values.yaml`	Default configurable values (replicas, image, service, resources)
`templates/`	Go-templated manifests — Deployment, Service — rendered at install time
`templates/_helpers.tpl` + `NOTES.txt`	Reusable label snippets, and post-install usage text

MENTAL MODEL

A chart is a parameterized set of manifests; `values.yaml` is the knob panel every template reads from.

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
resources: {}
```

The complete `values.yaml` from `webapp-chart/` — every template in this chart reads from these keys.

MANIFEST ANATOMY

A template, not a fixed manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
  labels:
    {{- include "webapp.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app.kubernetes.io/instance: {{ .Release.Name }}
  template:
    metadata:
      labels:
        {{- include "webapp.labels" . | nindent 8 }}
    spec:
      containers:
        - name: webapp
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 80
          {{- with .Values.resources }}
          resources:
            {{- toYaml . | nindent 12 }}
          {{- end }}
```

Every `{{ .Values.x }}` and `{{ .Release.Name }}` resolves from `values.yaml` (or a `--set/-f` override) at render time; `{{- with .Values.resources }}` skips the block entirely when unset; `_helpers.tpl` supplies the shared `webapp.labels` block. Same template, different values → a different rendered manifest.

DO IT — TWO WAYS

Quick overrides to explore, a values file to keep

QUICK — --SET FLAGS	VERSIONABLE — A VALUES FILE
`helm install my-web ./webapp-chart --set replicaCount=2`	`helm upgrade my-web ./webapp-chart -f values.yaml`
Great for a one-off override, nothing to save	You own the file — reviewable, re-appliable, diffable

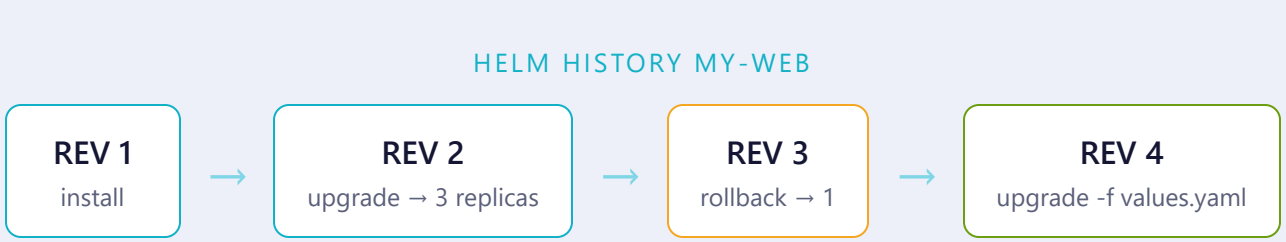
THE BRIDGE TRICK

Render locally before touching the cluster — Helm's answer to `--dry-run`:

```
$ helm template my-web ./webapp-chart \
    --set replicaCount=2 # prints manifests, applies nothing
```

SEE IT

Every action is a new, numbered revision




```
$ helm install my-web ./webapp-chart -n training --set replicaCount=2   # REV 1 · install
$ helm upgrade my-web ./webapp-chart -n training --set replicaCount=3   # REV 2 · scale to 3
$ helm rollback my-web 1 -n training                                    # REV 3 · rollback to REV 1
$ helm upgrade my-web ./webapp-chart -n training -f values.yaml          # REV 4 · values file
```

A rollback doesn't erase history or "go back in time" — it creates a **brand-new revision** whose content matches an older one. `helm history` only ever grows.

THE DECISION

Helm or Kustomize?

CHOOSE HELM	CHOOSE KUSTOMIZE
Packaging a reusable app — versioning, templating logic	Patching manifests you already own, per environment
Need install / upgrade / rollback tracked as revisions	`kubectl apply` is idempotent — no release state to track
Config is values-driven: `--set` or a values file	No templating language — strategic-merge or JSON patches on plain YAML
Distributed as a chart — a repo, or local like this lab	Built into `kubectl -k` — zero extra tooling to install

MENTAL MODEL

Kustomize **edits** YAML; Helm **renders and tracks** it as a release. Many teams use both — Helm for a chart, Kustomize to patch it further.

SAME CHANGE, BOTH WAYS

Run 3 replicas — patch vs. values

Kustomize — patch the YAML you own

```yaml
# overlays/prod/replica-patch.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp
spec:
  replicas: 3        # strategic-merge onto the base
```

Helm — set a value, the template fills in

```yaml
# values-prod.yaml   (helm upgrade -f)
replicaCount: 3

# templates/deployment.yaml consumes it:
#   replicas: {{ .Values.replicaCount }}
```

THE MENTAL-MODEL DIFFERENCE

Kustomize **patches** real, complete YAML — no placeholders. Helm **fills in** template variables at render time. Same 3 replicas, opposite philosophy.

GOTCHAS — THE ONES THAT BITE

Where people trip

- **Rollback grows history, it doesn't shrink it.** Rolling back to revision 1 becomes a new revision 3 — the bad revision 2 still shows up in `history`.
- **Forgetting `--namespace/-n`.** The release lands in (or upgrades) the wrong namespace silently.
- **`--set` for nested/list values gets fragile fast.** Past a key or two, switch to a values file.
- **Values layer, they don't replace.** Mixing `-f` files and `--set` flags — the last one on the command line wins per key.
- **A bad upgrade doesn't kill old Pods first.** Rolling-update protection keeps the last-good Pods running while the new ones fail — check before assuming the outage is total.
- **`helm uninstall`, not `delete`.** That's leftover Helm v2 naming — v3 has no Tiller and no `delete` verb.
- **`get values` wants a release, not a chart.** `helm get values my-web` — not `repo/chart`; the latter reports the release as not found.

YOUR HELM TOOLKIT

Commands to keep at hand

```
# discover & inspect a chart
$ helm search repo nginx      # find charts (hub = Artifact Hub)
$ helm show chart ./chart     # chart metadata
$ helm show values ./chart    # every overridable value
$ helm lint ./chart           # validate
$ helm template NAME ./chart  # render locally, no cluster

# install & upgrade
$ helm install NAME ./chart -n NS
$ helm upgrade --install NAME ./chart  # idempotent
$ helm upgrade NAME ./chart -f v.yaml --set k=v
#   + --dry-run  --atomic  --wait
```

```
# inspect a live release
$ helm list -n NS             # releases (-A = all ns)
$ helm status NAME -n NS      # state + NOTES.txt
$ helm get values NAME --all  # values in effect
$ helm get manifest NAME      # applied YAML
$ helm get all NAME           # everything about it

# history, rollback & remove
$ helm history NAME -n NS     # every revision
$ helm rollback NAME REV -n NS  # revert (new rev)
$ helm uninstall NAME -n NS   # --keep-history to retain
```

Tip: `helm get/status` take a **release name** (`helm get values my-web`), not `repo/chart`.

RECAP

Helm in one breath

- A chart = templates + `values.yaml` + `Chart.yaml` metadata
- `install` / `upgrade` / `rollback` each create a new numbered revision
- `--set` for a quick override, `-f` for versioned config
- Helm packages & tracks releases; Kustomize patches manifests you own

HANDS-ON

Now run `Lab 8.2`

`labs/8.2/`

Next: **8.3 — CRDs**